SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 10: Final Mini-Project Integration — Systems Engineering for Agriculture

---

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x and Google Colaboratory
Location: Morogoro, Tanzania

The hallmark of a professional engineer is the ability to synthesize disparate components into a unified, functional system. In this final module, we combine every concept learned in AGE 219 to build a comprehensive 'Smart Grain Bin Manager.' This project simulates a real-world post-harvest monitoring system, demonstrating the power of programming in food security and loss prevention.

Contents

1 Introduction: The Engineer as a System Integrator **2**

2 Theoretical Framework: Software Development for Engineers **2**
- 2.1 1. Requirements Analysis . . . 2
- 2.2 2. System Architecture Design . . . 2
- 2.3 3. Implementation (Coding) . . . 2
- 2.4 4. Verification and Validation . . . 2

3 Case Study Project: The Smart Grain Bin Manager **3**

4 Phase 1: Defining the System Architecture (OOP) **3**

5 Phase 2: Implementing Modular Logic (Functions) **3**

6 Phase 3: Data Ingestion and Simulation (Loops) **4**

7 Phase 4: Analytics and Visualization (Pandas & Matplotlib) **4**

8 The Full Integrated Script **5**

9 Summary and Course Reflection **6**
- 9.1 Key Lessons Learned . . . 6
- 9.2 Looking Ahead: The Digital Engineer . . . 6

10 Best Practices for Final Projects **7**

11 Glossary of Final Terms **8**

12 The Final Assessment: Building Your Own Manager **9**
- 12.1 Exercise 1: The Automated Poultry House . . . 9
- 12.2 Exercise 2: Advanced Statistics Integration . . . 9
- 12.3 Exercise 3: Hardware Fail-Safe Simulation . . . 9
- 12.4 Exercise 4: GUI Mockup (Markdown) . . . 9

1 Introduction: The Engineer as a System Integrator

Throughout this semester, we have dissected the world of computer programming into its constituent parts: the atoms (variables), the heartbeat (loops), the brain (logic), and the skeleton (data structures). However, in the School of Engineering and Technology, we do not build "parts." We build **Systems**.

A system is more than the sum of its parts. An automated tractor is not just a collection of code—it is an integration of path-planning algorithms, engine control logic, sensor data streams, and emergency fail-safes.

In this final lecture, we apply **Systems Thinking**. We will build a complete software package that models, monitors, and controls a grain storage facility. This project will require you to think like a Software Engineer while maintaining the rigor of a Post-Harvest Engineer.

2 Theoretical Framework: Software Development for Engineers

Before we write the first line of code for our project, we must understand the **Software Development Life Cycle (SDLC)**. In professional agricultural engineering, jumping straight into coding leads to expensive hardware failures.

2.1 1. Requirements Analysis

What must the system do? For a grain bin, the requirements are:

- Monitor internal temperature and moisture.
- Identify risks of spoilage or mold growth.
- Control aeration fans to stabilize the environment.
- Log all data for historical auditing.
- Visualize trends for the facility manager.

2.2 2. System Architecture Design

We will use an **Object-Oriented** design. We will create a `GrainBin` class that acts as the "Digital Twin." This object will contain all the attributes (state) and methods (logic) of a physical bin.

2.3 3. Implementation (Coding)

This is where we translate our logic into Python, utilizing the "Batteries Included" libraries like Pandas and Matplotlib.

2.4 4. Verification and Validation

Testing the system against simulated data to ensure the aeration fans turn on when the conditions are critical.

3 Case Study Project: The Smart Grain Bin Manager

The Problem: Grain spoilage in storage is a major cause of post-harvest loss in Tanzania. Spoilage is primarily driven by ”Hot Spots” (pockets of high temperature) and high moisture content, which encourage mold and insect activity.

The Solution: We will build a Python program that:

1. Defines a `GrainBin` class.
2. Simulates sensor data over a 24-hour period.
3. Implements an ”Aeration Logic” function.
4. Uses Pandas to store and analyze the logs.
5. Uses Matplotlib to generate an automated performance report.

4 Phase 1: Defining the System Architecture (OOP)

We start by modeling our hardware as a class. This allows us to scale easily; if a warehouse has 50 bins, we simply create 50 instances of the class.

```
class GrainBin:
    """Represents a physical grain storage unit."""

    def __init__(self, bin_id, capacity_tons, crop_type):
        self.bin_id = bin_id
        self.capacity = capacity_tons
        self.crop = crop_type

        # Initial Sensor States
        self.temperature = 25.0 # Celsius
        self.moisture = 13.0    # Percentage
        self.fan_active = False

        # Data storage for history
        self.history = []

    def update_sensors(self, temp, moisture):
        """Simulates receiving new data from IoT sensors."""
        self.temperature = temp
        self.moisture = moisture

    def get_status_message(self):
        status = "ON" if self.fan_active else "OFF"
        return f"Bin {self.bin_id}: Temp={self.temperature}C, Moist={self.moisture}%, Fan
    ={status}"
```

5 Phase 2: Implementing Modular Logic (Functions)

Engineering systems require thresholds. We create a specialized function to determine if aeration is required. This follows the **Modified Henderson Equation** logic in a simplified form.

```
def check_spoilage_risk(temp, moisture):
    """
    Evaluates risk based on temperature and moisture.
    Returns: 'CRITICAL', 'WARNING', or 'SAFE'
    """
```

```
    if temp > 35 or moisture > 15:
        return "CRITICAL"
    elif temp > 30 or moisture > 14:
        return "WARNING"
    else:
        return "SAFE"

def control_aeration(bin_object):
    """Logic to decide if fans should be toggled."""
    risk = check_spoilage_risk(bin_object.temperature, bin_object.moisture)

    if risk == "CRITICAL":
        bin_object.fan_active = True
        return "Fans activated due to high risk."
    elif risk == "SAFE" and bin_object.temperature < 20:
        bin_object.fan_active = False
        return "Fans deactivated: Conditions optimal."
    else:
        return "Maintaining current fan state."
```

6 Phase 3: Data Ingestion and Simulation (Loops)

In a real system, sensors report data every minute. We will simulate a 24-hour cycle using a `for` loop and random number generation.

```
import random

def simulate_day(bin_obj):
    print(f"--- Starting 24hr Simulation for Bin {bin_obj.bin_id} ---")

    for hour in range(24):
        # Simulate ambient temp rising during day and falling at night
        # Base temp + daily variation + random noise
        new_temp = 25 + (hour % 12) + random.uniform(-1, 1)
        # Simulate moisture creeping up slightly
        new_moisture = 13 + (hour * 0.05)

        # Update bin state
        bin_obj.update_sensors(new_temp, new_moisture)

        # Apply control logic
        msg = control_aeration(bin_obj)

        # Log the data for Phase 4
        bin_obj.history.append({
            'Hour': hour,
            'Temperature': new_temp,
            'Moisture': new_moisture,
            'Fan_Status': 1 if bin_obj.fan_active else 0
        })

        if hour % 6 == 0: # Print update every 6 hours
            print(f"Hour {hour}: {msg}")
```

7 Phase 4: Analytics and Visualization (Pandas & Matplotlib)

Once the simulation is complete, we use our analysis tools to generate a report.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

def generate_performance_report(bin_obj):
    # Convert list of dictionaries to DataFrame
    df = pd.DataFrame(bin_obj.history)

    # Calculate key metrics
    avg_temp = df['Temperature'].mean()
    max_moist = df['Moisture'].max()
    fan_hours = df['Fan_Status'].sum()

    print("\n--- PERFORMANCE SUMMARY ---")
    print(f"Average Storage Temp: {avg_temp:.2f} C")
    print(f"Maximum Moisture Peak: {max_moist:.2f} %")
    print(f"Total Fan Runtime: {fan_hours} hours")

    # Visualization
    fig, ax1 = plt.subplots(figsize=(12, 6))

    color = 'tab:red'
    ax1.set_xlabel('Hour of Day')
    ax1.set_ylabel('Temperature (C)', color=color)
    ax1.plot(df['Hour'], df['Temperature'], color=color, linewidth=2, label='Temp')
    ax1.tick_params(axis='y', labelcolor=color)

    ax2 = ax1.twinx() # Create a second Y-axis for moisture
    color = 'tab:blue'
    ax2.set_ylabel('Moisture (%)', color=color)
    ax2.plot(df['Hour'], df['Moisture'], color=color, linestyle='--', label='Moisture')
    ax2.tick_params(axis='y', labelcolor=color)

    plt.title(f"24-Hour Environmental Log: Bin {bin_obj.bin_id} ({bin_obj.crop})")
    plt.grid(alpha=0.3)
    plt.show()
```

8 The Full Integrated Script

Below is the complete integrated code. This represents the synthesis of the entire semester.

```
# --- AGE 219 FINAL PROJECT: SMART GRAIN BIN MANAGER ---
import pandas as pd
import matplotlib.pyplot as plt
import random

# 1. System Class
class GrainBin:
    def __init__(self, bin_id, capacity, crop):
        self.bin_id = bin_id
        self.capacity = capacity
        self.crop = crop
        self.temperature = 25.0
        self.moisture = 13.0
        self.fan_active = False
        self.history = []

    def update(self, t, m):
        self.temperature = t
        self.moisture = m
        self.history.append({
            'Temp': t, 'Moist': m, 'Fan': 1 if self.fan_active else 0
        })

```

```
# 2. Logic Functions
def apply_logic(bin_obj):
    if bin_obj.temperature > 32.0 or bin_obj.moisture > 14.5:
        bin_obj.fan_active = True
    elif bin_obj.temperature < 22.0:
        bin_obj.fan_active = False

# 3. Main Execution Block
def main():
    # Instantiate Bin
    my_bin = GrainBin("SUA-SILO-01", 1000, "Maize")

    # Simulate 48 hours of data
    for h in range(48):
        t = 24 + 10 * random.random() # Random temp between 24-34
        m = 13 + 0.1 * h # Gradual moisture increase
        my_bin.update(t, m)
        apply_logic(my_bin)

    # Analysis
    df = pd.DataFrame(my_bin.history)
    df.to_csv("silo_log.csv", index=False) # Exporting data (Module 7)

    # Plotting
    df[['Temp', 'Moist']].plot(figsize=(10,5), title="Silo Monitoring Report")
    plt.xlabel("Hours Elapsed")
    plt.ylabel("Value")
    plt.show()

if __name__ == "__main__":
    main()
```

9 Summary and Course Reflection

We have reached the end of AGE 219. We began with ”Hello, Farm!” and ended with a fully autonomous post-harvest control simulator.

9.1 Key Lessons Learned

- **Precision:** In engineering, a single syntax error can stop a machine.
- **Modularity:** Large problems are just collections of small problems solved by functions.
- **Data-Driven Design:** Good engineers don’t guess; they use Pandas and Matplotlib to prove their designs work.
- **Responsibility:** As we discussed in our ethical clauses, these tools must be used for the betterment of society and the protection of our food systems.

9.2 Looking Ahead: The Digital Engineer

The skills you have gained here are the prerequisite for:

- **AGE 316:** Microcomputer Systems.
- **AGE 328:** Computer Networking and Communications.
- **AGE 418:** Sensors and Controls in Precision Agriculture.

- **Final Year Projects:** Where you will likely build physical prototypes controlled by the Python logic you learned this semester.

10 Best Practices for Final Projects

1. **Robustness:** Always use `try-except` when reading external sensor files.
2. **Comments:** Your code is a document. Explain the ”Why,” not just the ”How.”
3. **Simplicity:** Do not use a complex class if a simple function will do. But if you have 10 identical machines, *always* use a class.
4. **Visualization:** Always check your axes and units before presenting a graph to a supervisor.

11 Glossary of Final Terms

- **API (Application Programming Interface):** A set of rules that allows your code to talk to other software or hardware.
- **Capstone Project:** A final task that integrates all previously learned skills.
- **Digital Twin:** A virtual representation of a physical object or system.
- **Fail-safe:** A design feature that in the event of a failure, responds in a way that will cause no or minimal harm to other devices or to personnel.
- **Integration:** The process of combining software components into a single system.
- **SDLC:** Software Development Life Cycle; the process used by the industry to design, develop, and test high-quality software.
- **Simulation:** The imitation of the operation of a real-world process or system over time.
- **System:** A set of interacting or interdependent component parts forming a complex/intricate whole.

12 The Final Assessment: Building Your Own Manager

To complete this course, you are tasked with modifying the ”Smart Grain Bin Manager” for a different agricultural scenario.

12.1 Exercise 1: The Automated Poultry House

Task: Rewrite the code to manage a Poultry House.

- **Inputs:** Ammonia level (ppm) and Temperature (°C).
- **Actuators:** Exhaust fans and Heaters.
- **Logic:** If ammonia > 25 ppm, turn on exhaust. If temp < 18°C, turn on heater.

12.2 Exercise 2: Advanced Statistics Integration

Using the `silo_log.csv` generated by the project: 1. Load the CSV in a separate notebook. 2. Use Pandas to find the hour with the **highest temperature**. 3. Calculate the **total percentage of time** the fan was active over the 48-hour period.

12.3 Exercise 3: Hardware Fail-Safe Simulation

Modify the `GrainBin` class to include a boolean attribute `sensor_malfunction`. **Task:** If `sensor_malfunction` is `True`, the `apply_logic` function must automatically turn the fan **ON** as a safety precaution, regardless of the temperature reading.

12.4 Exercise 4: GUI Mockup (Markdown)

In a text cell, describe how you would design a ”Farmer Dashboard” for this system. What three buttons and what two graphs would you prioritize for a user who is not an engineer?

CERTIFICATE OF MODULE COMPLETION

This concludes the formal lectures for AGE 219: Basics of Computer Programming. Congratulations on your journey into Engineering Informatics.

Dr. Kadeghe Fue, PhD, P.Eng (T)
Department of Agricultural Engineering, SUA